

体系结构Lab5

PB22111665 胡揚嘉

代码及其运行测试

1. 矩阵乘shi'xain代码

```
__global__ void Matmul3(float *A, float *B, float *C, unsigned N)
// 计算每个block的起始位置
// 每个block有64(T)个线程，默认一个Block解决T*S这样一个块的计算
// 反推出grid中的block数，第一维是N / U，第二维是N / T，默认可整除
// 每个thread的职责
// 循环N / S次完成任务
// 1. 将S个数据从A中取出，放入自己的register中
// 2. 合作将S*U个数据从B中取出，放入block的shared memory中，每个thread负责U个
    位的数据(因为这里S=4, U=16, S*U=T)
{
    // unsigned tx = threadIdx.x; // 这里我的Block维度是(1, 64)，所以tx=0。实际保
    证后面用不到这个tx数据
    unsigned ty = threadIdx.y; // ty也是标识目前的thread在block中的位置的唯一标识
    unsigned bx = blockIdx.x;
    unsigned by = blockIdx.y;
    unsigned row = by*blockDim.y + ty;
    // unsigned col = bx*blockDim.x + tx;
    __shared__ float Bsub[S][U];
    float Areg[S];
    float sum[U] = {0.0f};

    unsigned loop_count;
    for(loop_count = 0; loop_count < N; loop_count += S){
        // 一次循环中，赋值一次shared memory和register，同时计算一次矩阵乘法
        // 1. 将S个数据从A中取出，放入自己的register中

        #pragma unroll
        for(int i = 0; i < S; i++){
            Areg[i] = A[row*N + loop_count + i];
        }

        // 2. 合作将S*U个数据从B中取出，放入block的shared memory中，每个thread负责1
        个点位的数据(因为这里S=4, U=16, S*U=T)
        Bsub[ty / U][ty % U] = B[(loop_count + ty / U) * N + ty % U + bx *
    U];

    __syncthreads();
    // 3. 计算一行的点
    // #pragma unroll
    for(int i = 0; i < U; i++){
        // 对于负责的一行的每一点
        for(int j = 0; j < S; j++){
            // 计算出这一行的点
            sum[i] += Areg[j] * Bsub[j][i];
        }
    }
}
```

```

    }
    __syncthreads();
}
// 4. 将结果放入C中
for(int i = 0; i < U; i++){
    C[row*N + bx*U + i] = sum[i];
}
}

```

代码解析:

- 我的 grid 中的 block 数量是(N/U, N/T), 而 block 中的 thread 数量是(1, T)
- 对于每一个 block 需要完成一个T*U矩阵的乘法, 而对于每一个 thread, 需要完成一行U个位置的计算
- 每个线程的工作: (循环N/S次)
 - 将S个数据从 global mem 放置到 register
 - 每个 block 中所有的 thread 合作, 每个 thread 放置 $\frac{S \cdot U}{T} = 1$ 个元素到 shared mem 中
 - 计算(1,U)(U,1)的矩阵乘法, 得到一个结果, N/S次求和, 即为真实目标矩阵的一个点的结果
- 对于所有的 register 级别的循环操作, 使用 #pragma unroll 操作展开, 提高性能
- 在数据转移与计算之间, 需要使用 __syncthreads() 完成内存屏障。同样, 在N/S小份计算与最后求和之间, 需要有 `` 完成内存屏障。

2. 测试结果及分析

下面是我真实实验的配置和性能结果

```

#define Blocksize (32)
#define Matsize (4096)
#define Verifysize (1024)
#define T (64)
#define U (16)
#define S (T/U)

N:  4096  time1: 220.779098511  time2: 129.223144531  time3: 102.211441040

#define Matsize (4096)
#define Verifysize (1024)
#define T (64)
#define U (8)
#define S (T/U)

N:  4096  time1: 228.304718018  time2: 132.822448730  time3: 192.542633057

#define Blocksize (32)
#define Matsize (4096)
#define Verifysize (1024)
#define T (64)
#define U (4)
#define S (T/U)

N:  4096  time1: 237.374603271  time2: 128.459655762  time3: 343.517883301

```

+ 发现, 随着S的变大, 时间变长。这与我最初的预期是相反的。

- + 每个线程需要执行 N/S 次小矩阵相乘，得到的结果累和为最后的结果。
- + 那么实验 $S=1$ 时，看看情况怎么样

```
#define Blocksize (32)
#define Matsize (4096)
#define Verifysize (1024)
#define T (64)
#define U (64)
#define S (T/U)

N: 4096 time1: 214.284988403 time2: 124.956993103 time3: 42.935607910
```

- + 确实每次只算1个乘法，循环 N 次，反而快了。
- + 猜测是否是因为 $S=1$ 时，share memory的使用率更高了。
- + 试验增大 T ，保持 $T=U$

```
#define Blocksize (32)
#define Matsize (4096)
#define Verifysize (1024)
#define T (128)
#define U (128)
#define S (T/U)

N: 4096 time1: 230.449707031 time2: 132.513336182 time3: 36.778862000
```

```
#define Blocksize (32)
#define Matsize (4096)
#define Verifysize (1024)
#define T (256)
#define U (256)
#define S (T/U)

N: 4096 time1: 228.954910278 time2: 135.071868896 time3: 72.533760071
```

- + 发现 T 增大，性能没有再增加了，查看自己的GPU的配置
- + Registers per Block: 65536
- + 合理怀疑是寄存器的使用率过高了
- + 补充试验 $T=32$ ，研究规律

```
#define Blocksize (32)
#define Matsize (4096)
#define Verifysize (1024)
#define T (32)
#define U (32)
#define S (T/U)

N: 4096 time1: 225.183517456 time2: 132.310424805 time3: 84.149772644
```

- + 符合预期，

总结如下：

实验配置：

所有实验的 N 矩阵大小都是4096

实验编号	T	U	S=T/U
1	64	16	4
2	64	8	8
3	64	4	16
4	64	64	1
5	128	128	1
6	256	256	1
7	32	32	1

实验编号	time1 (ms)	time2 (ms)	time3 (ms)
1	220.779	129.223	102.211
2	228.304	132.822	192.542
3	237.374	128.459	343.517
4	214.285	124.957	42.936
5	230.450	132.513	36.779
6	228.955	135.072	72.534
7	225.183	132.310	84.150

分析：

- `time1`和`time2` 作为对照组，没有改变任何变量，随着实验不同，有略微变动，误差在 $\sigma_1 = 11.842$ (`time1`)和 $\sigma_2 = 3.734$ (`time2`)，能够接受，猜测`time3`实际上也有 $\sigma < \sigma_2 = 3.734$ 的误差，和实际不同配置下的时间差距很大，远远大过3.7几个数量级，故下面直接比较是由统计意义的
- 实验1-4，固定 `b1ock` 中的 `thread` 数量，变化U，探究结果的不同

直接发现，随着U增大，S减少，时间减少，当其为 `s=1` 时，效果最好

 - 后面的改变T实验中，固定`S=1`
 - 分析：
 - 一个 `thread` 的一个循环中，需要搬运S个register和1个shared mem，如果S过大，会让程序停留，等待 `__syncthreads()` 完成内存屏障
 - 每次循环，只需要计算一个乘法，否则线程会频繁等待，导致性能下降。
- 实验4-7：固定 `S=1`，变化 T
 - 当 `S=1` 时，`time3` 的性能显著提升。
 - 实验4（`T=64`）和实验5（`T=128`）的性能相近，且优于其他实验。
 - 实验6（`T=256`）和实验7（`T=32`）的性能较差，说明 T 过大或过小都会影响性能。
 - 分析：

- T 的选择需要平衡线程块的大小和共享内存的使用。过大的 T 可能导致共享内存不足，过小的 T 可能无法充分利用计算资源。

问题回答:

1. 请分析三种GPU矩阵乘法中对GPU global memory、shared memory、register的访问次数（忽略非矩阵元素的寄存器访问，如坐标索引变量row对寄存器的访问）

- 算法1：直接相乘，每个线程

global mem 访问 $3N+1$ 次

register 访问 $N+1$ 次

整个程序： N^2 个线程

global mem 访问 $N^2 \cdot (3N + 1)$ 次

register 访问 $N^2 \cdot (N + 1)$ 次

- 算法2：分块乘法，每个线程

global mem 访问 $2N/Blocksize+1$ 次

shared mem 访问 $2N/Blocksize + 2N$ 次

register 访问 $N+1$ 次

整个程序： N^2 个线程

global mem 访问 $N^2 \cdot (2N/Blocksize + 1)$ 次

shared mem 访问 $N^2 \cdot (2N/Blocksize + 2N)$ 次

register 访问 $N^2 \cdot (N + 1)$ 次

- 算法3：寄存器矩阵乘法，每个线程

global mem 访问 $(S+1)*N/S+U$ 次

shared mem 访问 $(U*S+1)*N/S$ 次

register 访问 $(S+2US)*N/S + U = (1+2U)*N+U$ 次

整个程序： N^2/U 个线程

global mem 访问 $N^2/U \cdot ((S + 1) * N/S + U)$ 次

shared mem 访问 $N^2/U \cdot ((U * S + 1) * N/S)$ 次

register 访问 $N^2/U \cdot ((1 + 2U) * N + U)$ 次

2. 寄存器矩阵乘法减少了线程数目及shared memory使用量，但增加了寄存器使用量。请结合课程所学分析这样做有何好处，有何坏处。（提示：从warp个数及block所需资源上分析）

好处

1. 减少线程数目：

- **提高线程利用率：**减少线程数目可以减少线程管理的开销，使得每个线程承担更多的计算任务，从而提高线程的利用率。

- **减少线程同步开销**：线程数目减少意味着需要同步的线程也减少，从而降低了线程同步的开销。这样加快了速度
- 2. **减少 shared memory 使用量**：
 - **降低 shared memory 竞争**：减少 shared memory 的使用量可以降低 shared memory 的访问冲突和竞争，提高 shared memory 的访问效率。
- 3. **增加寄存器使用量**：
 - **提高计算效率**：寄存器是 GPU 中**最快**的存储器，增加寄存器的使用量可以减少对 slower memory（如 global memory 或 shared memory）的访问，从而提高计算效率。

坏处

1. **增加寄存器使用量**：
 - **限制线程块大小**：每个线程块可以使用的寄存器数量是有限的，增加寄存器的使用量可能会限制每个线程块中的线程数目，从而影响并行度。本题中不存在，因为最大取到了 256，任然在GPU的资源内
- 实测资源情况：

```
(base) huyangjia@huyangjia:~/Coding/CS_Arch/comparch25spring-  
gem5/LAB5$ ./a.out  
Device 0: NVIDIA GeForce RTX 3060 Laptop GPU  
  Compute Capability: 8.6  
  MultiProcessor Count: 30  
  Shared Memory per Block: 49152 bytes  
  Registers per Block: 65536  
  Threads per Block: 1024  
  Max Threads Dim: (1024, 1024, 64)  
  Max Grid Dim: (2147483647, 65535, 65535)  
  Warp Size: 32  
  Max Threads per SM: 1536  
  Max Threads per Block: 1024  
  Clock Rate: 1425000 kHz  
  Total Constant Memory: 65536 bytes  
  Total Global Memory: 6441926656 bytes  
  regsPerBlock: 65536  
-----
```

- **增加寄存器溢出的风险**：如果寄存器使用量过大，可能会导致寄存器溢出，使得部分数据需要存储在 slower memory 中，从而降低性能。例如我在将 `T = thread` 从 128 调整到 256 时，速度明显下降，说明大部分 register 被缓存到 shared mem
2. **减少线程数目**：
 - **增加每个线程的计算负担**：每个线程需要承担更多的计算任务，可能会导致某些线程的计算负担过重，影响性能。这样限制了最大的 T 的大小，对于超大的 N 的矩阵，性能反而可能下降
 - **Warp 数量减少可能隐藏延迟的能力下降**
 - GPU 通过快速切换 warp 来隐藏内存访问延迟。如果 warp 总数减少，SM 可能没有足够的 warp 来切换，导致计算单元等待数据（也就是上课讲到的某些单元如运算单元空等），利用率下降。

3. 请结合课程所学从线程访存合并方面分析寄存器矩阵乘法对global memory的访问哪里可以继续优化。这一优化手段对基础矩阵乘法、分块矩阵乘法有效吗？请说明理由并给出优化位置。

原理：

- 关键在于，对于矩阵 $A[i][k]$ ，其随着k变化，内存访问是连续的
- 然而对于 $B[k][j]$ ，随着k变化，内存访问每次都是不连续的，每次会间隔 $N * \text{length}(\text{float})$ 内存位置。
- 所以，为了能够使用线程访存合并的方法解决问题，可以将矩阵B转置后，存储再 global memory 中。这样，对B的访问也能够享受到类似 cache 的好处了。

对于 **寄存器矩阵乘法**：

- 将需要访问的 shared mem 转置放置，这样对 shared mem 的访问的效率会更高
- 将B矩阵转置保存，这样搬运到 shared mem 时，访存命中更高

对于 **基础矩阵**：

- 再从CPU搬运矩阵到的GPU时，尝试将矩阵B转置保存，这样对 global mem 访问的命中率也会极大升高，利用率线程访存合并的优势

对于 **分块矩阵乘法**：

- 同上，将B矩阵装载时，转置保存，这样再搬运矩阵B到 shared mem 时，访存命中更高